NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

TECHNICAL REPORT:
EVENT MANAGEMENT SYSTEM (EMS)

Subject: Introduction to Databases
Class: DS66B - FDA
Instructor: Trần Hùng
Student: Nguyễn Bảo Khánh
Project Source (GitHub): https://github.com/baokhanh1410/event-management-system

May 3rd, 2026, Hanoi

TABLE OF CONTENT

1. Introduction

The Event Management System (EMS) is a comprehensive digital platform engineered to optimize organizational workflows, ensure financial precision, and integrate advanced Data Science methodologies to refine user engagement. The primary objective of the system is to streamline the lifecycle of an event, spanning from initial conception and resource planning to real-time operational execution and rigorous post-event analytical assessment.

1.1. Core System Features

The functional architecture of the EMS is categorized into several critical modules:

- **Event Lifecycle Management:** Facilitates the creation, modification, and logistical coordination of diverse event types.
- **Registration and Attendance Tracking:** Manages guest enrollment and provides real-time verification during event entry.
- **Financial Monitoring:** Enables comparative analysis between projected budgetary allocations and actual expenditures.
- **Role-Based Access Control (RBAC):** Implements granular permission levels for Admin, Staff, Organizer, and Guest personas.
- **Intelligent Recommendations:** Leverages an **`XGBoost-based`** model to provide personalized event suggestions.

1.2. Technology Stack

The EMS is built upon a robust technical foundation designed for scalability and performance:

- **Frontend Interface:** Developed using the **`Streamlit`** framework for dynamic web application delivery.
- **Backend Logic:** Implemented in `Python`, managing core business rules and system processes.
- **Database Management:** Utilizes **`MySQL`** for persistent, relational data storage.
- **Analytical Framework:** Employs **`XGBoost`** and **`Scikit-Learn`** for advanced predictive modeling.

2. Database Design (Logical)

The logical design of the EMS database, defined in **`schema.sql`**, prioritizes relational integrity and query efficiency. The architecture incorporates advanced SQL features to automate constraints and provide high-level abstractions for complex data aggregations.

2.1. Logical Schema Components

2.1.1. Relational Tables and Integrity

The system is founded upon a normalized relational model comprising core entities such as **`events`**, **`guests`**, **`organizers`**, and **`registrations`**. Data integrity is strictly maintained through foreign key constraints and unique indexes, ensuring consistent relationships across financial (**`event_finance`**) and administrative (**`roles`**, **`permissions`**) domains.

2.1.2. Server-Side Components

Operational efficiency is enhanced through pre-compiled SQL components:

- **Stored Procedures:** The **`sp_check_in_guest`** procedure centralizes the check-in logic, enforcing business rules to prevent data duplication.
- **Database Views:** Views such as **`vw_event_attendance_summary`** and **`vw_finance_summary`** provide abstracted, real-time metrics for attendance and profitability.

3. Implementation

The implementation phase integrates the logical design with the application layer, ensuring secure data persistence and an intuitive user interface.

3.1. System Architecture

3.1.1. Authentication and Security Module

Defined in **`auth.py`**, this module utilizes **`bcrypt`** for cryptographic hashing of user credentials. It manages the user lifecycle and enforces the RBAC framework by dynamically retrieving permissions based on assigned database roles.

3.1.2. Data Persistence and Connectivity

Database connectivity is managed via **`SQLAlchemy`** in **`connection.py`**. All CRUD operations are centralized in **`crud.py`**, utilizing a hybrid approach of ORM-like transactions and optimized raw SQL queries for data preparation using `Pandas`.

3.2. User Interface Design

The UI is architected into specific pages to serve distinct operational needs:

- **Access Gate (`0_Login.py`)**: Handles secure user entry and stateful session management.
- **Event Nexus (`1_Events.py`)**: Provides permission-aware views for discovery and administrative management.
- **Operational Hub (`2_Attendance.py`)**: Facilitates real-time check-ins and attendance analytics.
- **Economic Analytics (`3_Finance.py`)**: Delivers advanced KPI visualizations and budgetary oversight tools.

4. Results

The deployment of the EMS resulted in a highly functional platform capable of handling complex event workflows. Key achievements include:

- Seamless real-time guest check-in via secure stored procedures.
- Dynamic financial dashboards providing automated profit/loss assessments.
- High-accuracy event recommendations powered by optimized ML models.
- Strict enforcement of security protocols through hashed credentials and RBAC.

5. Conclusion

The Event Management System successfully demonstrates the synergy between relational database design and modern application logic. By integrating predictive analytics with robust administrative controls, the project provides a scalable framework for organizational efficiency. Future enhancements will focus on expanding the ML feature set and introducing automated resource procurement modules.